

Claviger - Rétrospective des interventions humaines V1.1

Développement assisté par IA : qui produit le code, qui dirige la production ?

Version documentaire : V1.1 - analyse du processus de développement
Anonymisé pour publication dans un dépôt public

13 septembre 2026

Résumé exécutif

Claviger est développé avec une assistance IA intensive. L'assistant peut proposer des architectures, écrire des fichiers complets, produire des tests, analyser des erreurs et suggérer des migrations. Cette réalité n'est ni masquée ni minimisée. Elle ne signifie toutefois pas que le projet est développé de manière autonome par l'IA.

L'analyse des échanges de développement montre un modèle plus proche d'une chaîne de production collaborative : l'IA accélère fortement la production de code, tandis que le développeur fixe les objectifs, challenge les hypothèses, impose les invariants, décide du niveau de qualité attendu, exécute les tests, contrôle les commits, dirige les changements d'architecture et assume la validation avant déploiement.

Thèse du document

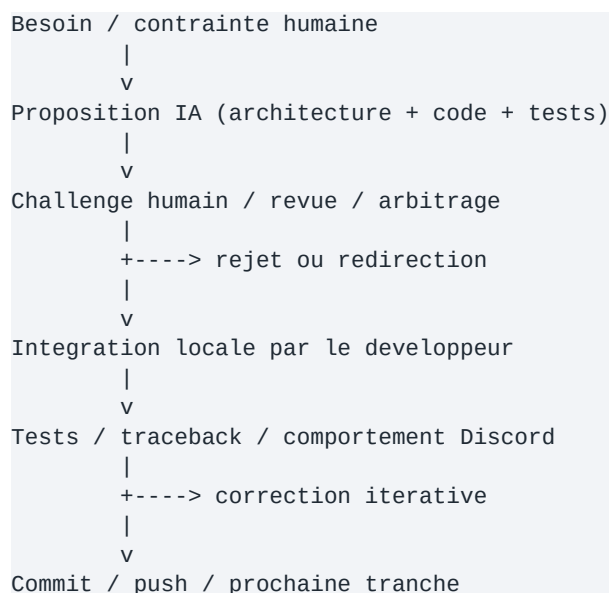
Le rôle du développeur ne disparaît pas ; il se déplace progressivement. Une part du temps auparavant consacrée à la frappe et à la syntaxe est transférée vers la spécification, l'architecture, le review, le test, l'observabilité, l'intégration et la gouvernance technique.

1. Ce que ce document cherche à montrer

- Documenter honnêtement l'usage intensif de l'IA dans la production du code.
- Identifier les interventions humaines ayant réellement changé la direction du projet.
- Montrer que le prompt initial ne suffit pas à expliquer l'architecture finale.
- Distinguer production de code, décision d'architecture, validation et responsabilité de mise en production.
- Illustrer un mode de développement plausible en 2026 où la compétence du développeur se déplace vers la conduite de systèmes de production assistée par IA.

Ce document n'essaie pas d'attribuer chaque ligne à un auteur précis. Dans un workflow itératif, une fonction peut être proposée par l'IA, corrigée sur demande du développeur, refactorisée à nouveau par l'IA puis validée ou rejetée après tests. Une comptabilité ligne par ligne donnerait une représentation trompeuse de la collaboration.

2. Modèle de collaboration observé



L'IA peut donc être le principal producteur de texte source pendant certaines sessions sans être pour autant le principal décideur. Le pouvoir de direction se lit dans la capacité à arrêter une trajectoire, à imposer un invariant, à refuser une abstraction, à demander un fichier complet plutôt qu'un patch, à changer une stratégie de persistance ou à décider qu'une feature doit être repoussée.

3. Interventions humaines ayant redirigé le projet

3.1 Refus du code jetable et exigence de modularité

Très tôt, le développeur a refusé une logique de prototype rapide destiné à être réécrit plus tard. Cette intervention a poussé le projet vers des services réutilisables, des planners/executors/coordinators et une séparation entre l'interface Discord et la logique métier.

Décision humaine	Effet sur la production
Refuser un bot monolithique centré sur les handlers Discord	Création de services métiers indépendants de l'UI.
Préférer des modules réutilisables	Les futures commandes IA ou Web peuvent viser les mêmes services.
Accepter un coût initial de structure	Moins de refactor destructif lors des versions suivantes.

3.2 Challenge permanent des hypothèses de l'assistant

Le workflow ne se résume pas à « demander du code puis le copier ». Les propositions sont régulièrement contestées sur leur périmètre, leur lisibilité ou leurs conséquences. Plusieurs corrections structurantes viennent précisément d'un désaccord ou d'une remarque humaine.

- Une estimation erronée du nombre attendu de tests a été détectée grâce au résultat réel de pytest, puis corrigée au lieu d'être rationalisée.
- Un problème d'import circulaire a été isolé à partir du traceback et corrigé en remettant le modèle au rôle minimal attendu.
- Lorsque le nombre de micro-patches dans test_bot.py est devenu trop élevé, le développeur a refusé cette ergonomie et imposé le remplacement du fichier complet.
- Après un incident antérieur où un fragment de migration avait fait perdre une autre instruction SQL cohérente, la règle « ne jamais remplacer partiellement un bloc de migration » a été élevée au rang de convention de production.

Ces exemples sont importants : ils montrent qu'un développeur assisté par IA doit rester capable de lire un traceback, de comprendre les dépendances et de juger la sûreté d'une méthode d'intégration. La vitesse de génération ne remplace pas le contrôle.

3.3 Passage du single-guild au vrai multi-guild

L'une des redirections majeures de la V1.1 consiste à refuser une simple compatibilité multi-serveurs superficielle. Le développeur a demandé que l'application puisse servir plusieurs guilds sans partager leur configuration par accident. Cette demande a forcé une séparation conceptuelle forte entre identité de l'application, identité locale de la guild, ownership de la base et readiness propre à chaque serveur.

```
1 application Discord
-> 1 identité application
-> 1 base SQLite
-> N guilds
    -> N identités locales
    -> N configurations
    -> N readiness states
```

Cette redirection n'est pas un détail d'implémentation. Elle modifie le modèle mental du runtime et prépare la suppression de dépendances historiques telles qu'un DISCORD_GUILD_ID global.

3.4 Distinction « DB prête » / « guild prête »

Un risque architectural a été explicitement identifié : une base saine et déjà configurée pour une première guild ne doit pas entraîner l'exposition automatique de workflows sur un second serveur fraîchement rejoint. Le développeur a validé une architecture où la santé de l'application et la readiness de chaque guild sont évaluées séparément.

```
DB READY
!=
Guild READY
```

La conséquence directe est qu'une nouvelle guild reçoit uniquement les capacités de recovery/configuration nécessaires, tandis qu'une guild déjà configurée conserve son fonctionnement normal. Cette décision réduit les fuites de configuration et les comportements surprenants entre serveurs.

3.5 Refus des inférences sémantiques risquées

Dans la configuration ADMIN, l'assistant aurait pu être tenté de déduire qu'un forum nommé « activity » est destiné à l'activité et qu'un autre est destiné aux erreurs. Le développeur a imposé une règle plus robuste : lorsque la structure existante est ambiguë, les noms et l'ordre ne suffisent pas à attribuer une sémantique. La décision doit être explicitement validée par l'administrateur.

- Les IDs Discord deviennent l'identité stable après configuration.
- Les noms restent cosmétiques et réparables.
- IMPORT et NEEDS_CHOICE ne doivent pas inventer un mapping sémantique.
- La convergence doit privilégier la sécurité et l'explicabilité à l'automatisation aveugle.

3.6 Le développeur impose aussi l'ergonomie de maintenance

Une partie importante du rôle de direction ne concerne pas directement les features. Elle concerne la manière dont le code est livré et maintenu. Pendant la V1.1, le développeur a formalisé deux règles nouvelles : fournir un fichier complet lorsque plusieurs zones changent, et documenter systématiquement les fonctions significatives et les décisions non triviales.

Règle imposée	Motif
Fichier complet si plusieurs changements corrélés	Réduire les erreurs de patch manuel et garder un état cohérent.
Imports groupés par responsabilité	Navigation et remplacement plus rapides.
Docstrings avec Args / Returns / Raises	Rendre le contrat des fonctions exploitable indépendamment de la conversation.
Commentaires sur le pourquoi	Conserver la logique des décisions de sécurité et de compatibilité.
Compatibilité temporaire explicitement marquée	Faciliter le nettoyage avant release.

3.7 L'audit est utilisé comme outil de direction, pas comme décoration

Le développeur a demandé un audit critique du projet et a ensuite utilisé ses conclusions pour redéfinir la roadmap. Les constats sur l'architecture, la sécurité, les logs, l'observabilité ou la préparation au multi-guild ne sont pas restés dans un PDF séparé : ils sont progressivement transformés en tranches de développement, en garde-fous et en conventions.

Ce mécanisme est caractéristique d'un rôle de développeur qui évolue : le travail ne consiste plus seulement à implémenter directement une fonction, mais aussi à organiser plusieurs agents ou outils de production autour d'un cycle de critique, de correction et de vérification.

4. Ce que fait réellement l'assistant IA

- Inspection du repository et comparaison avec l'état attendu.
- Proposition d'architectures et de tranches de migration.
- Production de fichiers Python complets.
- Écriture et adaptation de tests unitaires / runtime.
- Diagnostic de tracebacks fournis par le développeur.
- Proposition de commandes de validation Ruff / pytest.
- Rédaction de documentation, prompts maîtres et audits.
- Préparation de titres et descriptions de commits.

Dire que l'IA « code » est donc exact dans ce projet : elle produit effectivement une part substantielle du code proposé. Ce terme doit cependant être distingué de « diriger le projet » ou « mettre en production de façon autonome ».

5. Ce que conserve le développeur

Responsabilité	Manifestation concrète
Vision produit	Décide ce que le bot doit devenir et quelles features sont prioritaires.
Architecture	Challenge les couplages, refuse le jetable, impose les invariants multi-guild.
Sécurité	Choisit les comportements fail-closed et refuse les inférences risquées.
Review	Lit les propositions, signale les zones fragiles et demande des reprises complètes.
Intégration	Copie/modifie le code localement, résout les conflits éventuels et contrôle les fichiers réellement changés.
Validation	Exécute Ruff, pytest et les smoke tests Discord.
Versioning	Décide quand une tranche est cohérente, commit, push et merge.
Production	Contrôle les migrations, sauvegardes, déploiements et retours arrière.

6. Pourquoi le challenge humain reste indispensable

L'IA peut produire très vite une solution localement plausible. Le risque principal n'est donc plus seulement l'erreur syntaxique ; il devient l'erreur d'hypothèse : mauvais périmètre, abstraction prématurée, état global caché, migration incomplète, comportement Discord supposé plutôt que vérifié, test qui couvre la mauvaise chose ou modification trop large pour être intégrée sans risque.

- Un assistant peut oublier un invariant discuté plusieurs heures plus tôt.
- Il peut proposer un changement correct localement mais incohérent avec une migration ou un déploiement réel.
- Il peut sous-estimer la difficulté d'un patch manuel et provoquer des erreurs d'intégration humaines évitables.
- Il peut produire une explication convaincante d'un résultat faux ; le test réel reste supérieur à la narration.
- Il peut optimiser pour la solution immédiate alors que le développeur possède une vision plus longue du produit.

Conséquence

La compétence critique du développeur devient encore plus importante lorsque la génération devient plus rapide. Plus l'IA peut produire de code, plus la qualité du cadrage, du review et des critères d'acceptation détermine la qualité finale du système.

7. Évolution du rôle du développeur en 2026

Le cas Claviger suggère une évolution progressive plutôt qu'une substitution brutale. Le développeur peut écrire moins de code caractère par caractère tout en assumant davantage de responsabilités de conception et de contrôle.

Rôle historiquement dominant	Rôle renforcé avec assistance IA
Écrire chaque fonction à la main	Spécifier le contrat puis reviewer une proposition générée.
Chercher longtemps la syntaxe ou les APIs	Vérifier que l'API proposée correspond réellement au contexte et à la version.
Refactorer mécaniquement plusieurs fichiers	Définir le refactor, contrôler son périmètre et valider les invariants.
Écrire manuellement tous les tests	Définir les scénarios critiques et juger si les tests générés prouvent réellement le comportement.
Produire la documentation a posteriori	Imposer que le code et les docstrings conservent directement les décisions.
Être principalement exécutant du code	Devenir davantage architecte, reviewer, opérateur et responsable de qualité.

Cette évolution ne rend pas la compréhension du code facultative. Au contraire, pour challenger efficacement une IA, le développeur doit pouvoir reconnaître une mauvaise séparation des responsabilités, un état mutable dangereux, une transaction mal bornée, une erreur de lifecycle ou un test trompeur. L'assistance déplace donc la compétence plus qu'elle ne l'annule.

8. Production du code vs direction de la production

L'assistant IA peut produire :

- classes
- services
- repositories
- tests
- documentation
- migrations proposées

```
Le developpeur dirige :  
- le besoin  
- les priorites  
- les invariants  
- les compromis  
- l'acceptation ou le rejet  
- l'integration  
- les tests reels  
- les commits  
- le deploiement
```

Cette distinction permet de décrire le projet sans minimiser l'IA et sans effacer le rôle humain. Le code peut être majoritairement généré pendant une session donnée, tout en restant produit sous une direction humaine explicite.

9. Les tests comme frontière d'autorité

Dans ce workflow, un message de l'assistant disant « cela devrait fonctionner » n'a pas valeur de validation. La frontière d'autorité est le comportement observé : tests ciblés, Ruff, suite pytest complète puis, lorsque pertinent, validation sur l'environnement Discord DEV.

```
proposition IA  
-> integration locale  
-> tests cibles  
-> Ruff  
-> pytest complet  
-> smoke Discord si necessaire  
-> commit seulement si coherent
```

Cette discipline donne au développeur un levier simple pour conserver la maîtrise : l'IA peut proposer autant de code qu'elle veut, mais aucune tranche n'est considérée comme acquise avant que le résultat concret soit vérifié.

10. L'importance de la mémoire architecturale

Une longue collaboration avec une IA produit elle-même une nouvelle forme de dette : des décisions peuvent rester uniquement dans la conversation. La demande récente de commentaires systématiques, de docstrings et de prompts maîtres de continuité répond directement à ce problème.

- Le dépôt doit expliquer son architecture sans dépendre d'un historique de chat.
- Les compatibilités temporaires doivent être visibles dans le code pour pouvoir être supprimées.
- Les audits doivent devenir des éléments de backlog et pas seulement des documents statiques.
- Les prompts maîtres documentent le cadre de collaboration et les invariants à un instant donné.

11. Limites et honnêteté méthodologique

Cette rétrospective ne démontre pas qu'un même modèle de collaboration convient à tous les projets. Elle ne démontre pas non plus qu'une IA puisse être laissée seule face à des décisions de production critiques. Elle décrit un cas réel de développement assisté où le développeur choisit volontairement de déléguer une grande partie de la production de code tout en renforçant les mécanismes de direction et de validation.

- L'IA peut se tromper techniquement.
- Le développeur peut également se tromper dans ses arbitrages.
- Les tests peuvent avoir des angles morts.
- Une architecture qui paraît saine en DEV doit encore être confrontée au multi-guild réel, aux migrations et à l'exploitation.
- Le résultat reste un logiciel à auditer, observer et maintenir, pas une preuve automatique de qualité parce qu'il a été produit avec une IA.

12. Conclusion

Claviger illustre un changement concret de métier : la production de code peut devenir une ressource abondante, interactive et rapide. Ce qui devient rare est la capacité à donner une direction cohérente à cette production, à conserver des invariants sur plusieurs semaines, à savoir quand refuser une solution séduisante, à organiser les tests et à rendre le système exploitable par d'autres.

Dans ce modèle, le développeur n'est ni réduit à un « prompt engineer » ni remplacé par un générateur. Il devient le chef d'orchestre d'une chaîne de production logicielle où l'IA est un contributeur extrêmement productif. Son travail se rapproche progressivement de l'architecture, du review, de la qualification, de l'intégration et de l'exploitation - sans perdre la nécessité de comprendre le code suffisamment profondément pour challenger celui qui le produit.

Formulation synthétique

L'IA écrit une part importante du code. Le développeur décide quel logiciel est en train d'être construit, sous quelles contraintes, avec quels tests, et à quel moment il est assez fiable pour devenir une nouvelle base de travail.